YamanRedTeam

Uncensored Local LLM for Cybersecurity Research

Local LLM Setup Guide

Independently built and documented by YamanRedTeam

1. Introduction

What is a Local LLM?

A local large language model (LLM) is an AI language model that is downloaded and executed directly on your own hardware, instead of being called over the network as a hosted API. All the computation — reading your prompt and generating a response — happens on your CPU or GPU, and the model weights live on your disk.

What is Ollama?

Ollama is a local model runtime that makes it simple to download, manage, and run open-weight language models on your own machine. It exposes a small REST API (by default on `http://localhost:11434`) that any application — including this one — can call to get a completion from a locally-loaded model, without any cloud dependency.

What is an uncensored/open model?

Most commercial chat assistants are tuned with heavy built-in refusal behavior. "Uncensored" or lightly-aligned community models — such as the Dolphin family used by this project — are fine-tuned to be more direct and technical, with fewer blanket refusals. This makes them useful for detailed technical work, but it does not make them more accurate, more capable, or a substitute for professional judgment. They are still ordinary language models that can be wrong, and they should be used only for authorized research, education, CTFs, and controlled lab environments.

Why run an LLM locally?

- **Privacy** — prompts and responses never leave your machine.
- **No API costs** — no per-token billing or rate limits.
- **Offline capability** — works without an internet connection once the model is downloaded.
- **Full control** — you choose the model, the system prompt, and how the data is handled.

Why local AI is useful for cybersecurity research

Security research often involves sensitive material — internal logs, code from private repositories, findings that aren't yet public, or lab data tied to an ongoing engagement. Sending that content to a third-party cloud API may not be appropriate or permitted. Running the assistant locally keeps that data on the researcher's own machine while still providing fast, direct technical explanations for concepts, code, and log analysis.

Dolphin model overview

Dolphin is a family of open, community-fine-tuned language models built on top of base models such as Llama 3 and Mixtral. It is distributed as GGUF weights and runs well through Ollama on consumer hardware. This project defaults to `dolphin-llama3`, but any Ollama-compatible model can be substituted via configuration.

2. System Requirements

- Linux (this guide targets Kali Linux), macOS, or WSL2 on Windows
- Python 3.10 or newer
- 8 GB RAM minimum for small quantized models; 16 GB+ recommended
- 5–8 GB free disk space per model
- Optional: a discrete GPU for faster inference (CPU-only works, just slower)

3. Installing Ollama

```
# Official install script
curl -fsSL https://ollama.com/install.sh | sh

# Verify installation
ollama --version
```

Start the Ollama service (skip if it's already running as a system service):

```
ollama serve
```

4. Installing the Model

```
# Pull the default model used by this project
ollama pull dolphin-llama3

# Confirm it is installed
ollama list
```

> You can substitute any compatible Ollama model — e.g. `dolphin-mixtral`, `dolphin-phi`, `llama3`, `mistral` — by pulling it and changing `OLLAMA_MODEL` in your `.env` file. The model name is never hardcoded in the application source.

5. Python & Project Setup

Clone the repository

```
git clone https://github.com/<your-username>/Local-LLM-YamanRedTeam.git
cd Local-LLM-YamanRedTeam
```

Create a virtual environment

```
python3 -m venv venv
source venv/bin/activate
```

Install dependencies

```
pip install -r requirements.txt
```

Configure environment variables

```
cp .env.example .env
```

Variable	Default	Purpose
OLLAMA_MODEL	dolphin-llama3	Model tag Ollama should run
OLLAMA_HOST	http://localhost:11434	Ollama API base URL
APP_HOST / APP_PORT	0.0.0.0 / 8000	FastAPI bind address
REQUEST_TIMEOUT_SECONDS	120	Timeout for one generation
MAX_MESSAGE_LENGTH	6000	Max characters per message
MAX_HISTORY_MESSAGES	40	Max messages per conversation

6. FastAPI Architecture

The backend (`app.py`) is a small, stateless FastAPI service with four routes: `GET /` renders the chat UI, `GET /api/health` is a liveness probe, `GET /api/status` checks Ollama connectivity and model availability, and `POST /api/chat` validates an incoming conversation with Pydantic, prepends a fixed system prompt, and forwards it to Ollama's native `/api/chat` endpoint over HTTP using `httpx`, with a configurable timeout and structured error handling for connection failures, timeouts, and missing models.

7. Running the Application

Using the helper script (starts Ollama if needed, then the web app):

```
./run.sh
```

Or manually:

```
ollama serve &
source venv/bin/activate
uvicorn app:app --host 0.0.0.0 --port 8000
```

Then open **http://localhost:8000** in a browser.

8. Troubleshooting

Symptom	Likely cause / fix
Status shows "Ollama offline"	Run `ollama serve`, confirm `OLLAMA_HOST` is correct
Status shows "Model not pulled"	Run `ollama pull <OLLAMA_MODEL>`
First response is slow	Normal — model weights are loading into memory for the first time
Port already in use	Change `APP_PORT` in `.env`
ModuleNotFoundError on startup	Ensure the virtual environment is activated and `pip install -r requirements.txt` completed

9. Project Structure

```
Local-LLM-YamanRedTeam/
├── app.py
├── requirements.txt
├── run.sh
├── README.md
├── .gitignore
├── LICENSE
├── .env.example
├── templates/
│   └── index.html
├── static/
│   ├── css/style.css
│   ├── js/app.js
│   └── assets/yamanredteam-logo.svg
├── docs/
│   ├── Architecture.md
│   └── Local_LLM_Setup_YamanRedTeam.pdf
└── screenshots/
```

10. GitHub Setup

```
git init
git add .
git commit -m "Initial commit: YamanRedTeam local LLM"
git branch -M main
git remote add origin https://github.com/<your-username>/Local-LLM-YamanRedTeam.git
git push -u origin main
```

The project's `.gitignore` excludes virtual environments, `.env` files, `__pycache__`, downloaded model weights, and other local artifacts, so none of that is pushed to GitHub.

11. Responsible Use

> This project is for authorized security research, CTFs, and lab environments only. Only use it against systems you own or have explicit, written authorization to test. "Uncensored" describes the model's refusal behavior, not a license to target third-party systems or cause real-world harm. You are responsible for complying with all applicable laws and the terms of any engagement you operate under.